

Memoization Logic Tree Dry Run

Let's build the intuition first, then see the **memoization recursion tree** for $n = 3$, $k = 1$.

1. What is the main idea?

We need permutations of:

```
1, 2, 3, ..., n
```

having exactly k inverse pairs.

An inverse pair is:

```
i < j
nums[i] > nums[j]
```

For example:

```
[3, 1, 2]
(3,1) → inverse
(3,2) → inverse
Total = 2
```

2. The key observation: add numbers one by one

Suppose we already know all permutations of:

```
1 ... n-1
```

Now we want to insert n .

Since n is the **largest number**, the number of new inversions depends entirely on where we insert it.

For example, suppose:

```
1 2
```

We want to insert 3.

Insert at the end

```
1 2 3
```

3 has no smaller element after it.

```
New inversions = 0
```

Insert one position from the end

```
1 3 2
```

$3 > 2$

```
New inversions = 1
```

Insert at the beginning

```
3 1 2
```

$3 > 1$

$3 > 2$

```
New inversions = 2
```

So when inserting n , we can create:

```
0, 1, 2, ..., n-1
```

new inverse pairs.

3. Define the DP state

Let:

```
f(n, k)
```

mean:

Number of permutations using numbers $1 \dots n$ having exactly k inverse pairs.

Now, when inserting n , suppose it creates i new inversions.

Then the previous permutation of $1 \dots n-1$ must already have:

```
k - i
```

inverse pairs.

Therefore:

```
f(n, k) =
f(n-1, k)
+ f(n-1, k-1)
+ f(n-1, k-2)
+ ...
+ f(n-1, k-(n-1))
```

We only consider valid values where:

```
k - i >= 0
```

So:

```
C++
f(n, k) = Σ f(n - 1, k - i)
where i = 0 to min(k, n - 1)
```

4. Base cases

Case 1: $k == 0$

There is exactly **one** permutation with zero inverse pairs:

```
1 2 3 ... n
```

Therefore:

```
C++
f(n, 0) = 1
```

For example:

```
f(3, 0) = 1
[1, 2, 3]
```

Case 2: $n == 0$

If there are no numbers:

```
f(0, 0) = 1
```

But:

```
f(0, k > 0) = 0
```

In the recursive implementation, we can simply use:

```
C++
if (n == 0)
    return k == 0;
```

5. Memoization code

```
C++
class Solution {
public:
    static const int MOD = 1e9 + 7;

    int solve(int n, int k, vector<vector<int>>& dp) {
        // Base case
        if (k == 0)
            return 1;

        if (n == 0)
            return 0;

        // Already calculated
        if (dp[n][k] != -1)
            return dp[n][k];

        long long ans = 0;

        // Insert n and create i new inversions
        for (int i = 0; i <= min(k, n - 1); i++) {
            ans = (ans + solve(n - 1, k - i, dp)) % MOD;
        }

        return dp[n][k] = ans;
    }

    int kInversePairs(int n, int k) {
        vector<vector<int>> dp(n + 1,
                               vector<int>(k + 1, -1));

        return solve(n, k, dp);
    }
};
```

6. Now let's dry run $n = 3, k = 1$

We call:

```
solve(3, 1)
```

Meaning:

Using numbers {1,2,3}, how many permutations have exactly 1 inverse pair?

When inserting 3, it can create:

```
0, 1, or 2
```

new inversions.

But we only need total:

```
k = 1
```

So we only try:

```
i = 0
i = 1
```

We don't try $i = 2$, because:

```
k - i = 1 - 2 = -1 ❌
```

Therefore:

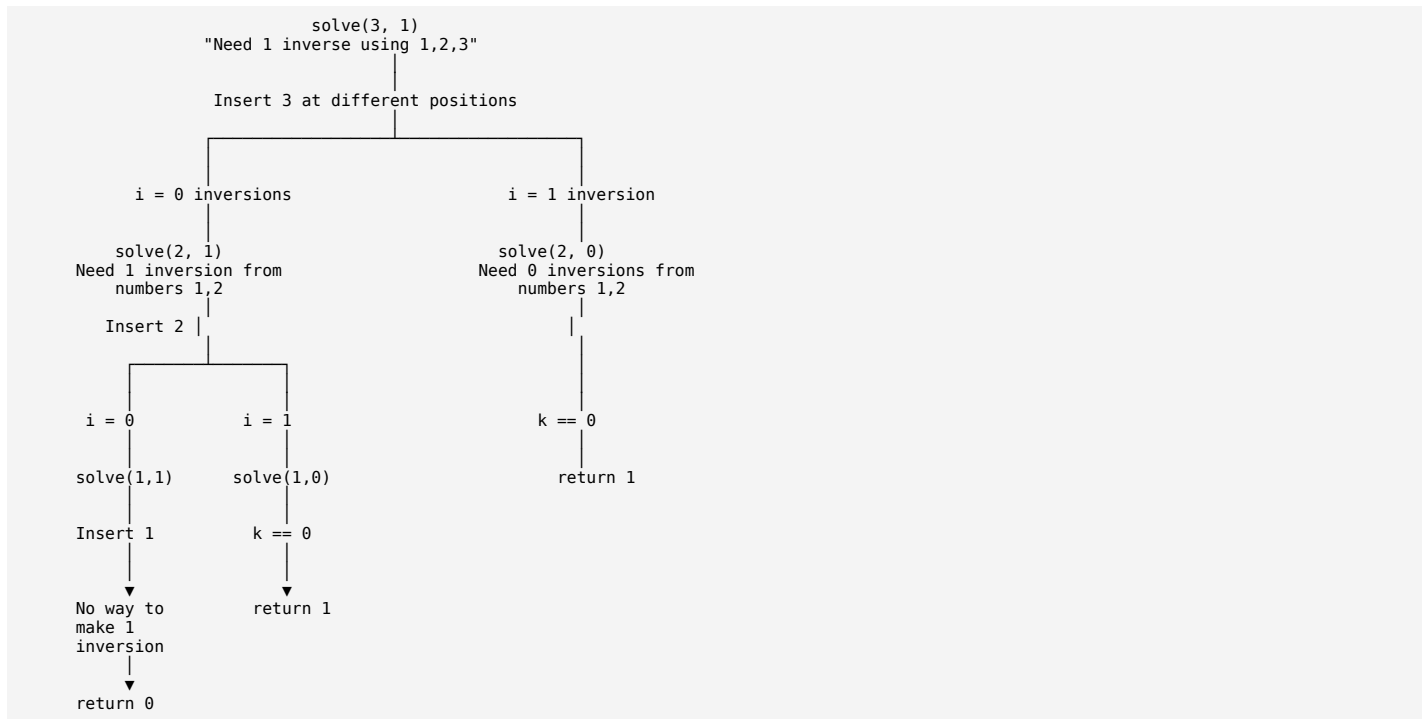
```
solve(3,1)
├── +---- insert 3 creating 0 inversions
│   │       └── solve(2,1)
│   └── +---- insert 3 creating 1 inversion
│       │       └── solve(2,0)
```

Mathematically:

```
f(3,1)
= f(2,1)
+ f(2,0)
```

Now let's expand both branches.

7. Complete recursion tree



Therefore:

```

solve(3,1)
= solve(2,1) + solve(2,0)
= [solve(1,1) + solve(1,0)] + 1
= [0 + 1] + 1
= 2
  
```

8. Let's understand what every branch actually represents

Root: solve(3, 1)

We have:

```
1, 2, 3
```

We need exactly:

```
1 inverse pair
```

Branch 1: Insert 3 and create 0 inversions

For 3 to create zero inversions, it must be inserted at the end:

```
_ _ 3
```

So the remaining numbers:

```
1, 2
```

must themselves produce:

```
1 inverse pair
```

Thus:

```
solve(2, 1)
```

The only permutation is:

```
[2, 1]
```

Now put **3** at the end:

```
[2, 1] + 3
= [2, 1, 3]
```

Inversions:

```
2 > 1 
```

3 creates no new inversions.

Total:

```
1 inversion
```

Contribution:

```
1
```

Diagram:

```
solve(2,1) = 1
  [2,1]
   |
   | insert 3
   | creates 0 inversions
   v
  [2,1,3]
Inverse pairs:
(2,1)
Total = 1
```

9. Branch 2: Insert 3 and create 1 inversion

Now **3** must create exactly:

```
1 new inversion
```

That means there must be exactly **one number after 3**.

So its position is:

```
_ 3 _
```

The smaller permutation must already have:

```
k - i
=
1 - 1
=
0
```

inversions.

Therefore:

```
solve(2, 0)
```

The only permutation with zero inversions is:

```
[1, 2]
```

Now insert **3** so that exactly one element comes after it:

```
[1, 3, 2]
```

The new inversion created is:

```
3 > 2 
```

So:

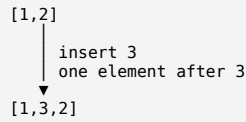
$\text{solve}(2,0) = 1$

Contribution:

1

Diagram:

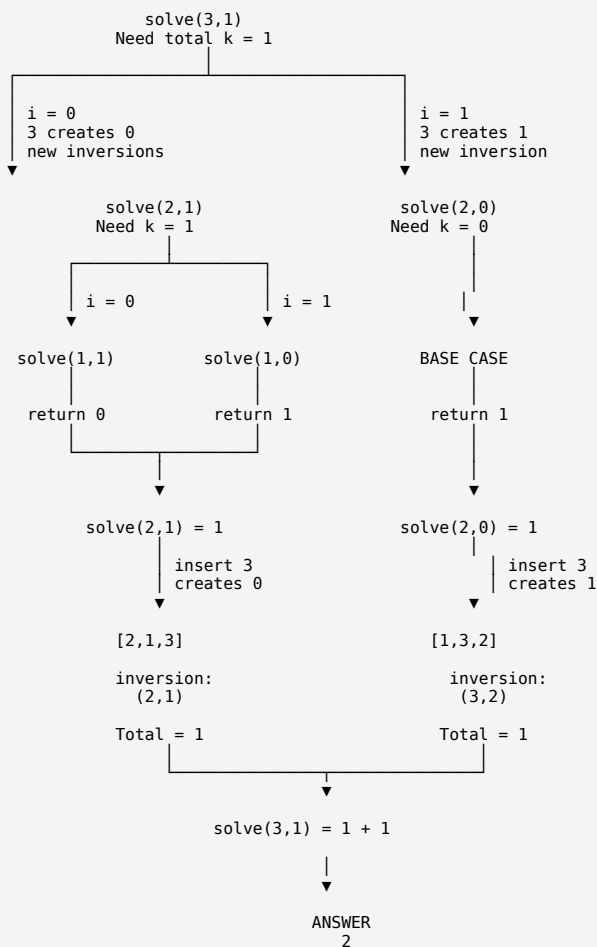
$\text{solve}(2,0) = 1$



New inverse pair:
(3, 2)

Total = 1

10. Final tree with the actual permutations



The most important thing to remember

At every recursive call:

$\text{solve}(n, k)$

we ask:

Where should I place n ?

If placing n creates i new inversions, then the remaining $n-1$ numbers must create:

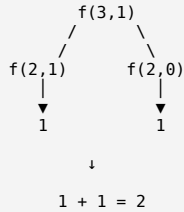
$k - i$

inversions.

So the core recurrence is:

```
C++
for (int i = 0; i <= min(k, n - 1); i++) {
    ans += solve(n - 1, k - i);
}
```

For $n = 3$, $k = 1$:



The two valid arrays are:

```
[2,1,3] → inverse pair (2,1)
[1,3,2] → inverse pair (3,2)
```

So the final answer is:

2

The **memoization solution is intuitive but has $O(n \times k \times \min(n, k))$ time complexity**, which can be too slow for the maximum constraints. The next step is to optimize this same recurrence using a **prefix sum / sliding-window DP**, reducing it to **$O(n \times k)$** .

Absolutely. Let's convert the **same memoization logic** into tabulation **step by step**, without changing the recurrence first.

1. Start with the memoization code

Our recursive function was:

```
C++
int solve(int n, int k, vector<vector<int>>& dp) {
    // Base case
    if (k == 0)
        return 1;

    if (n == 0)
        return 0;

    // Already calculated
    if (dp[n][k] != -1)
        return dp[n][k];

    long long ans = 0;
    for (int i = 0; i <= min(k, n - 1); i++) {
        ans = (ans + solve(n - 1, k - i, dp)) % MOD;
    }

    return dp[n][k] = ans;
}
```

The state is:

```
dp[n][k]
= number of permutations of 1...n
  having exactly k inverse pairs
```

The recursion is:

```
solve(n, k)
  depends on
  solve(n-1, k)
  solve(n-1, k-1)
  solve(n-1, k-2)
  ...
```

This dependency is the key to converting it into tabulation.

2. Remove the recursion

In memoization, we write:

```
C++
solve(n - 1, k - i, dp)
```

But in tabulation, we have already calculated the previous values.

So:

```
C++
solve(n - 1, k - i, dp)
```

becomes:

```
C++
dp[n - 1][k - i]
```

Therefore, this recursive part:

```
C++
for (int i = 0; i <= min(k, n - 1); i++) {
    ans += solve(n - 1, k - i, dp);
}
```

becomes:

```
C++
for (int i = 0; i <= min(k, n - 1); i++) {
    ans += dp[n - 1][k - i];
}
```

So our recurrence is now directly translated into tabulation.

3. What should the DP table look like?

We create:

```
C++
vector<vector<int>> dp(n + 1, vector<int>(k + 1, 0));
```

Rows represent:

number of elements

Columns represent:

number of inverse pairs

For example, for:

```
n = 3
k = 3
```

our table conceptually looks like:

	inverse pairs			
	0	1	2	3
n = 0				
n = 1				
n = 2				
n = 3				

Each cell:

dp[i][j]

means:

Using numbers $1 \dots i$, how many permutations have exactly j inverse pairs?

4. Convert the base cases

In recursion, we had:

```
C++
if (k == 0)
    return 1;
```

Meaning:

For every n ,
there is exactly one permutation
having 0 inverse pairs.

That permutation is always sorted:

1 2 3 ... n

Therefore:

```
C++
for (int i = 0; i <= n; i++) {
    dp[i][0] = 1;
}
```

So the first column becomes:

	k			
	0	1	2	3
$n = 0$	1	0	0	0
$n = 1$	1	0	0	0
$n = 2$	1	0	0	0
$n = 3$	1	0	0	0

Notice:

$dp[0][0] = 1$

This is also valid because there is one way to arrange zero elements with zero inverse pairs.

5. Now fill the table

Look at our recurrence:

```
C++
dp[n][k]
=
dp[n-1][k]
+ dp[n-1][k-1]
+ dp[n-1][k-2]
+ ...
```

So for every:

current number of elements = i
current inverse pairs = j

we do:

```
C++
for (int x = 1; x <= n; x++) {
    for (int inv = 1; inv <= k; inv++) {
        long long ans = 0;
        for (int newInv = 0;
             newInv <= min(inv, x - 1);
             newInv++) {
            ans += dp[x - 1][inv - newInv];
        }
        dp[x][inv] = ans % MOD;
    }
}
```

This is the direct conversion.

6. Let's build the table for $n = 3$, $k = 1$

Initially:

	k = 0	k = 1
n = 0	1	0
n = 1	1	0
n = 2	1	0
n = 3	1	0

Step 1: Fill $n = 1$

We calculate:

`dp[1][1]`

The recurrence says:

`dp[1][1]`
`=`
`dp[0][1]`

Because when inserting 1, it can create only:

0 inversions

Since:

`dp[0][1] = 0`

Therefore:

`dp[1][1] = 0`

Table:

	0	1
n = 0	1	0
n = 1	1	0
n = 2	1	0
n = 3	1	0

Makes sense:

Only permutation:

[1]

It has 0 inverse pairs.

So there is no permutation with 1 inverse pair.

7. Fill $n = 2$

We calculate:

`dp[2][1]`

Using the recurrence:

`dp[2][1]`
`= dp[1][1] + dp[1][0]`

Why two values?

Because when inserting 2, it can create:

0 new inversions
 or

1 new inversion

Case 1

2 creates 0 inversions:

$dp[1][1]$

We still need 1 inversion before inserting 2.

$dp[1][1] = 0$

Case 2

2 creates 1 inversion:

$dp[1][0]$

The previous array needs:

$1 - 1 = 0$ inversions

And:

$dp[1][0] = 1$

Therefore:

$$\begin{aligned} dp[2][1] &= \\ &= 0 + 1 \\ &= 1 \end{aligned}$$

Table becomes:

	0	1
n = 0	1	0
n = 1	1	0
n = 2	1	1
n = 3	1	0

The one permutation is:

[2, 1]

8. Fill n = 3

Now calculate:

$dp[3][1]$

Using exactly the same recurrence:

$$\begin{aligned} dp[3][1] &= \\ &= dp[2][1] + dp[2][0] \end{aligned}$$

Option 1: 3 creates 0 inversions

We need:

1 inversion already present

So:

$dp[2][1] = 1$

This represents:

[2, 1]

Insert 3 at the end:

```
[2,1,3]
```

Still:

```
1 inversion
```

Option 2: 3 creates 1 inversion

The previous permutation must have:

```
1 - 1 = 0 inversions
```

So:

```
dp[2][0] = 1
```

This represents:

```
[1,2]
```

Insert 3 so one element comes after it:

```
[1,3,2]
```

New inversion:

```
3 > 2
```

So this also gives exactly 1 inversion.

Therefore:

```
dp[3][1]
= dp[2][1] + dp[2][0]
= 1 + 1
= 2
```

Final table:

```

      inverse pairs
      0      1
n = 0  +-----+
      | 1 | 0 |
n = 1  +-----+
      | 1 | 0 |
n = 2  +-----+
      | 1 | 1 |
n = 3  +-----+
      | 1 | 2 |
      +-----+

```

Answer:

```
dp[3][1] = 2
```

9. Direct conversion: memoization → tabulation

Memoization

```

C++
int solve(int n, int k) {
    if (k == 0)
        return 1;
    if (n == 0)
        return 0;
    if (dp[n][k] != -1)
        return dp[n][k];
    long long ans = 0;
    for (int i = 0; i <= min(k, n - 1); i++) {
        ans += solve(n - 1, k - i);
    }
    return dp[n][k] = ans;
}

```

```
}
```

Step-by-step conversion

Step A: Function parameters become loops

```
C++
solve(n, k)
```

becomes:

```
C++
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= k; j++) {
    }
}
```

Step B: Recursive call becomes table lookup

```
C++
solve(n - 1, k - x)
```

becomes:

```
C++
dp[i - 1][j - x]
```

Step C: Keep the same transition

```
C++
for (int x = 0; x <= min(j, i - 1); x++) {
    ans += dp[i - 1][j - x];
}
```

10. Final tabulation code — direct conversion

```
C++
class Solution {
public:
    int kInversePairs(int n, int k) {

        const int MOD = 1e9 + 7;

        // dp[i][j]
        // = number of permutations of 1...i
        // having exactly j inverse pairs
        vector<vector<int>>> dp(
            n + 1,
            vector<int>(k + 1, 0)
        );

        // Base case:
        // Exactly one way to have 0 inverse pairs:
        // [1, 2, 3, ..., i]
        for (int i = 0; i <= n; i++) {
            dp[i][0] = 1;
        }

        // Build the table
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= k; j++) {
                long long ans = 0;

                // Insert i and create x new inversions
                for (int x = 0;
                     x <= min(j, i - 1);
                     x++) {
                    ans += dp[i - 1][j - x];
                    ans %= MOD;
                }

                dp[i][j] = ans;
            }
        }

        return dp[n][k];
    }
};
```

The complete mental connection

MEMOIZATION

```
solve(n, k)
    |
    | choose how many new inversions
    | are created by inserting n
    v
solve(n-1, k)
solve(n-1, k-1)
solve(n-1, k-2)
...
```

becomes:

TABULATION

```
dp[n][k]
    |
    | values from the previous row
    v
dp[n-1][k]
dp[n-1][k-1]
dp[n-1][k-2]
...
```

So:

```
Memoization
    |
    |
    v
solve(i-1, j-x)
    |
    | replace recursive call
    v
Tabulation
    |
    v
dp[i-1][j-x]
```

The important point is: **the logic has not changed at all**. We only changed **recursive calls into previously computed table values**.

One issue remains: this direct tabulation has **three nested loops**, giving roughly:

```
O(n * k * min(n, k))
```

which is too slow for $n, k = 1000$.

The next optimization keeps this exact DP meaning but uses a **prefix sum / sliding window** to reduce it to $O(n * k)$.

Yes. The reason is that **memoization removes repeated states, but each state itself still does a loop**.

Your state is:

```
C++
solve(n, k)
```

There are at most:

```
n * k = 1000 * 1000 = 10^6
```

different states.

That part is okay.

The problem is this loop

```
C++
for (int i = 0; i <= min(k, n - 1); i++) {
    ans = (ans + solve(n - 1, k - i, dp)) % MOD;
}
```

For **every DP state**, you may loop up to n times.

So complexity becomes approximately:

```
Number of states * work per state
O(n * k) * O(n)
= O(n^2 * k)
```

With:

```
n = 1000
k = 1000
```

worst case:

```
1000 × 1000 × 1000
= 10^9 operations
```

That is why you get **TLE**.

Visualizing the problem

For one state:

```
solve(1000, 1000)
```

you do:

```
solve(999, 1000)
solve(999, 999)
solve(999, 998)
solve(999, 997)
...
solve(999, 1)
```

Almost:

```
1000 function calls / lookups
```

Then for other states:

```
solve(999, 1000) → again loops ~999 times
solve(999, 999)  → again loops ~999 times
solve(999, 998)  → again loops ~999 times
```

Even though memoization prevents recalculating the **state**, it does **not** remove the transition loop.

Your recurrence

Your recurrence is:

```
dp[n][k]
=
dp[n-1][k]
+ dp[n-1][k-1]
+ dp[n-1][k-2]
+ ...
+ dp[n-1][k-(n-1)]
```

For example:

```
dp[5][6]
=
dp[4][6]
+ dp[4][5]
+ dp[4][4]
+ dp[4][3]
+ dp[4][2]
```

Every cell is calculating a range sum again and again.

For example:

```
dp[5][6]
= dp[4][6] + dp[4][5] + dp[4][4] + dp[4][3] + dp[4][2]

dp[5][7]
= dp[4][7] + dp[4][6] + dp[4][5] + dp[4][4] + dp[4][3]
```

Notice that most values are repeated.

```
dp[4][6]
dp[4][5]
dp[4][4]
dp[4][3]
┌──┴──┐ calculated again and again
```

This is where the optimization comes from.

Direct memoization complexity

Your code:

```
C++
for (int i = 0; i <= min(k, n - 1); i++) {
    ans = (ans + solve(n - 1, k - i, dp)) % MOD;
}
```

has:

```
Time: O(n × k × min(n, k))
Space: O(n × k) + recursion stack
```

For maximum constraints:

```
O(1000 × 1000 × 1000)
= O(109)
```

✗ Too slow.

The main optimization idea

Instead of calculating:

```
dp[i][j]
=
dp[i-1][j]
+ dp[i-1][j-1]
+ dp[i-1][j-2]
+ ...
```

from scratch every time, compare two consecutive cells.

Suppose:

```
dp[i][j]
=
dp[i-1][j]
+ dp[i-1][j-1]
+ ...
+ dp[i-1][j-(i-1)]
```

Now:

```
dp[i][j-1]
=
dp[i-1][j-1]
+ dp[i-1][j-2]
+ ...
+ dp[i-1][j-i]
```

Compare them.

To go from:

```
dp[i][j-1]
```

to:

```
dp[i][j]
```

we:

Add

```
dp[i-1][j]
```

Remove

```
dp[i-1][j-i]
```

So the optimized formula becomes:

```
C++
dp[i][j] =
dp[i][j-1]
+ dp[i-1][j]
- (j >= i ? dp[i-1][j-i] : 0);
```

This removes the inner loop completely.

Optimized tabulation code — $O(n \times k)$

```
C++
class Solution {
public:
    static const int MOD = 1e9 + 7;

    int kInversePairs(int n, int k) {
        vector<vector<int>> dp(
            n + 1,
            vector<int>(k + 1, 0)
        );

        // Base case:
        // Exactly one way to have 0 inverse pairs
        for (int i = 0; i <= n; i++) {
            dp[i][0] = 1;
        }

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= k; j++) {
                long long ans = dp[i - 1][j];

                // Add previous answer
                ans += dp[i][j - 1];

                // Remove the value that goes outside
                // the window of size i
                if (j >= i) {
                    ans -= dp[i - 1][j - i];
                }

                // Handle negative modulo
                ans = (ans + MOD) % MOD;

                dp[i][j] = ans;
            }
        }

        return dp[n][k];
    }
};
```

The key difference is:

Your code

```
C++
for (int x = 0; x <= min(k, n - 1); x++) {
    ans += solve(n - 1, k - x, dp);
}
```

↑ An inner loop for every state.

Optimized DP

```
C++
dp[i][j] =
    dp[i][j - 1]
    + dp[i - 1][j]
    - dp[i - 1][j - i];
```

↑ Only $O(1)$ work per state.

So:

```
Your memoization:
O(n × k × min(n,k)) ❌

Optimized DP:
O(n × k) ✅
```

For this problem, the **most important next step is understanding exactly how**

```
dp[i][j-1] + dp[i-1][j] - dp[i-1][j-i]
```

is derived from your original memoization loop. That is the bridge between your current code and the optimal solution.

Yes. This is the **most important part** of the optimization. Let's fill an actual DP row and see how the **cumulative/sliding sum** works.

We will use:

```
i = 3
```

That means we are filling the row:

```
dp[3][j]
```

using the previous row:

```
dp[2][j]
```

Step 1: Previous row

For numbers:

```
1, 2
```

the possible permutations are:

```
[1,2] → 0 inversions
[2,1] → 1 inversion
```

So:

```
dp[2][0] = 1
dp[2][1] = 1
dp[2][2] = 0
dp[2][3] = 0
dp[2][4] = 0
...
```

Let's say our previous row is:

j	0	1	2	3	4	5
dp[2][j]	1	1	0	0	0	0

Now we want to build:

```
dp[3][j]
```

Step 2: What does dp[3][j] mean?

When inserting **3**, it can create:

```
0, 1, or 2
```

new inversions.

Why?

Because **3** can be placed:

```
3 1 2 → 2 new inversions
1 3 2 → 1 new inversion
1 2 3 → 0 new inversions
```

Therefore:

```
dp[3][j]
=
dp[2][j]
+ dp[2][j-1]
+ dp[2][j-2]
```

So for every cell, we are taking a **window of 3 values from the previous row**.

Step 3: Fill dp[3][0]

Base case:

```
dp[3][0] = 1
```

Because:

```
[1,2,3]
```

is the only permutation with **0** inversions.

Table:

j	0	1	2	3	4
dp[3][j]	1	1	0	0	0

```
dp[2] 1 1 0 0 0
dp[3] 1 ? ? ? ?
```

Step 4: Fill dp[3][1]

Original recurrence says:

```
dp[3][1]
=
dp[2][1]
+
dp[2][0]
```

So:

```
= 1 + 1
= 2
```

Therefore:

```
j      0  1  2  3  4
dp[2] 1  1  0  0  0
dp[3] 1  2  ?  ?  ?
```

The two permutations are:

```
[2,1,3]
[1,3,2]
```

Now see the optimized formula

The code is:

```
C++
long long ans = dp[i - 1][j];
ans += dp[i][j - 1];
if (j >= i) {
    ans -= dp[i - 1][j - i];
}
dp[i][j] = ans;
```

For:

```
i = 3
j = 1
```

we do:

```
C++
ans = dp[2][1];
```

So:

```
ans = 1
```

Then:

```
C++
ans += dp[3][0];
```

So:

```
ans = 1 + 1
= 2
```

Since:

```
j >= i
1 >= 3 ❌
```

we don't subtract anything.

Therefore:

```
dp[3][1] = 2
```

So this:

```
C++
dp[i - 1][j]
+
dp[i][j - 1]
```

becomes:

```
dp[2][1] + dp[3][0]
= 1 + 1
= 2
```

Step 5: Fill dp[3][2]

Let's first use the **original recurrence**.

Since 3 can create 0, 1, 2 inversions:

```
dp[3][2]
=
dp[2][2]
+
dp[2][1]
+
dp[2][0]
```

Substitute values:

```
= 0 + 1 + 1
= 2
```

Now see the optimized calculation.

We already know:

```
dp[3][1] = 2
```

The code does:

```
C++
ans = dp[2][2];
ans += dp[3][1];
```

So:

```
ans = 0 + 2
= 2
```

Since:

```
2 >= 3 ❌
```

we don't subtract.

Therefore:

```
dp[3][2] = 2
```

Table:

j	0	1	2	3	4
dp[2]	1	1	0	0	0
dp[3]	1	2	2	?	?

Now the important part: Why does this work?

Look carefully.

We already calculated:

```
dp[3][1]
=
dp[2][1]
+
```

```
dp[2][0]
```

Diagram:

```
dp[3][1]
  dp[2][1]  dp[2][0]
    ↓        ↓
    [1]    +   [1]
          = 2
```

Now we want:

```
dp[3][2]
```

Original recurrence:

```
dp[3][2]
=
dp[2][2]
+
dp[2][1]
+
dp[2][0]
```

Notice:

```
      Already inside dp[3][1]
      ↓
dp[3][1] = dp[2][1] + dp[2][0]
```

So instead of recalculating those two:

```
dp[2][1]
+
dp[2][0]
```

we reuse:

```
dp[3][1]
```

Then just add the new value:

```
dp[2][2]
```

Therefore:

```
dp[3][2]
=
dp[3][1]
+
dp[2][2]
```

That is:

```
2 + 0 = 2
```

This is the **cumulative sum idea**.

Visualize it as a moving window

Previous row:

index	0	1	2	3	4	5
dp[2]	1	1	0	0	0	0

For $i = 3$, every answer takes at most **3 values**.

dp[3][0]

Window:

```
[1]
```

```
dp[3][0] = 1
```

dp[3][1]

Window moves:

```
[1] [1]
```

```
dp[3][1] = 1 + 1 = 2
```

dp[3][2]

Window moves again:

```
[1] [1] [0]
```

```
dp[3][2] = 1 + 1 + 0 = 2
```

Now the window has reached its maximum size:

```
3
```

because inserting 3 can create at most:

```
0, 1, 2
```

inversions.

So the window size can never exceed 3.

Step 6: Fill dp[3][3]

Original recurrence:

```
dp[3][3]
=
dp[2][3]
+
dp[2][2]
+
dp[2][1]
```

Notice carefully:

```
dp[2][0]
```

is no longer included.

Why?

Because our window has moved.

For dp[3][2], we had:

```
dp[2][2]
+
dp[2][1]
+
dp[2][0]
```

Now for dp[3][3], we need:

```
dp[2][3]
+
dp[2][2]
+
dp[2][1]
```

Let's compare:

```
dp[3][2]:
dp[2][2] + dp[2][1] + dp[2][0]
                        ↑
                    remove this
```

Move window right

```

dp[3][3]:
dp[2][3] + dp[2][2] + dp[2][1]
  ↑
add this

```

So:

```

NEW ANSWER
=
OLD ANSWER
+ new value
- old value leaving the window

```

Therefore:

```

dp[3][3]
=
dp[3][2]
+
dp[2][3]
-
dp[2][0]

```

Substitute:

```

= 2
+ 0
- 1

= 1

```

And that exactly matches the code:

```

C++
ans = dp[i - 1][j];

```

For $i = 3$, $j = 3$:

```

ans = dp[2][3]
     = 0

```

Then:

```

C++
ans += dp[i][j - 1];

```

So:

```

ans = 0 + dp[3][2]
     = 0 + 2
     = 2

```

Now:

```

j >= i
3 >= 3 ✓

```

Therefore:

```

C++
ans -= dp[i - 1][j - i];

```

Substitute:

```

ans -= dp[2][3 - 3]
ans -= dp[2][0]
ans = 2 - 1
ans = 1

```

Therefore:

```

dp[3][3] = 1

```

This is the exact cumulative/sliding window

```

Previous row: dp[2]
index      0      1      2      3      4
values    -----
          1      1      0      0      0

dp[3][0]
      [ 1 ]
sum = 1

dp[3][1]
      [ 1      1 ]
sum = 2

dp[3][2]
      [ 1      1      0 ]
sum = 2

dp[3][3]
           [ 1      0      0 ]
sum = 1

```

Notice the transition:

```

Window for dp[3][2]:
      [ dp[2][0], dp[2][1], dp[2][2] ]
= 1 + 1 + 0
= 2

```

Move one step:

```

Remove dp[2][0] ✗
      [ dp[2][1], dp[2][2] ]
Add dp[2][3] ✓
      [ dp[2][1], dp[2][2], dp[2][3] ]

```

So:

```

new sum
=
old sum
- dp[2][0]
+ dp[2][3]

```

Exactly:

```

C++
dp[i][j]
=
dp[i][j - 1]
+ dp[i - 1][j]
- dp[i - 1][j - i];

```

Fill the complete row for i = 3

Let's fill more values.

```

Previous row:
j      0      1      2      3      4      5
dp[2]  -----
        1      1      0      0      0      0

```

Now:

dp[3][0]

```
= 1
```

dp[3][1]

```
= dp[2][1] + dp[3][0]
```



```
= 1 + 1
= 2
```

dp[3][2]

```
= dp[2][2] + dp[3][1]
= 0 + 2
= 2
```

dp[3][3]

```
= dp[2][3]
+ dp[3][2]
- dp[2][0]
= 0 + 2 - 1
= 1
```

dp[3][4]

```
= dp[2][4]
+ dp[3][3]
- dp[2][1]
= 0 + 1 - 1
= 0
```

dp[3][5]

```
= dp[2][5]
+ dp[3][4]
- dp[2][2]
= 0 + 0 - 0
= 0
```

Final:

j	0	1	2	3	4	5
dp[2]	1	1	0	0	0	0
dp[3]	1	2	2	1	0	0

Which is correct because permutations of [1,2,3] have inversion counts:

```
[1,2,3] → 0
[1,3,2] → 1
[2,1,3] → 1
[2,3,1] → 2
[3,1,2] → 2
[3,2,1] → 3
```

Therefore:

```
0 inversions → 1 permutation
1 inversion → 2 permutations
2 inversions → 2 permutations
3 inversions → 1 permutation
```

So:

```
dp[3] = [1, 2, 2, 1]
```

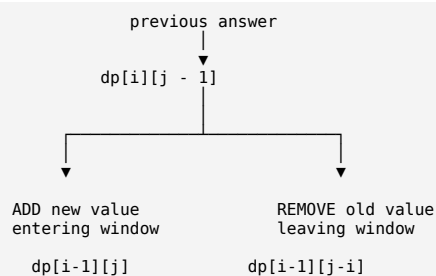
The simplest way to remember the code

This:

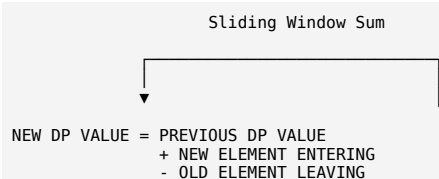
```
C++
long long ans = dp[i - 1][j];
ans += dp[i][j - 1];

if (j >= i) {
    ans -= dp[i - 1][j - i];
}
```

means:



So the entire logic is:



That is why this optimized DP is called a **cumulative/sliding-window optimization**: instead of summing the whole range again for every $dp[i][j]$, you reuse the previous sum, **add one new value**, and when the window becomes too large, **remove one old value**.

ChatGPT can make mistakes. Check important info.